任务 1：数据库与查询条件基础说明

以 DVWA 成绩查询 / 用户查询页面为实际场景说明：

1.页面输入为什么和数据库查询有关

网页里的用户查询、成绩查询、账号登录等功能，所有账号信息、用户数据、内容资料都存放在后台数据库中。网页本身无法储存大量数据，必须依靠接收用户输入的内容，去数据库中检索对应数据，才能展示内容，所以页面输入必然会和数据库查询产生关联。

2.最小业务链路

输入查询条件 → 拼接生成数据库 SQL 查询语句 → 数据库执行查询 → 网站接收数据 → 页面渲染并返回展示结果。

3.用户输入是如何进入查询条件的

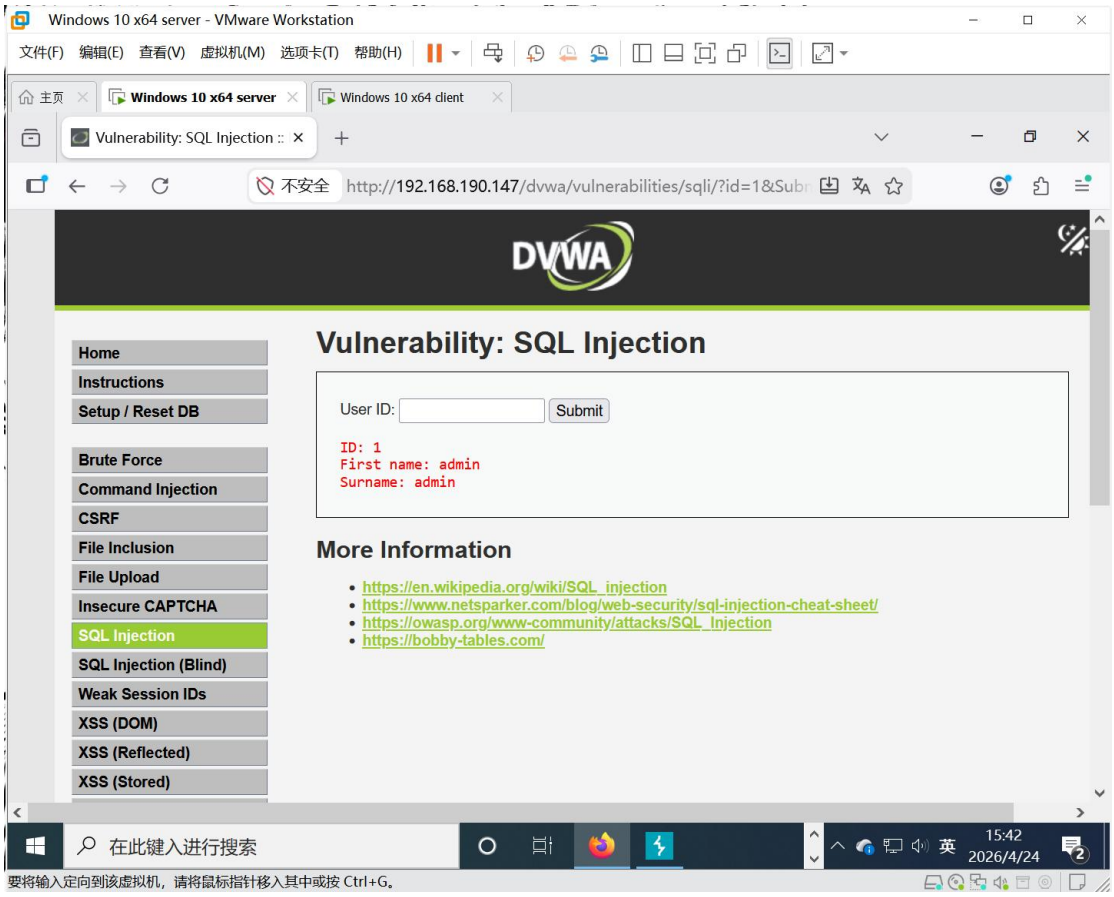
用户在网页输入框填写内容（如 id、用户名），前端把输入内容提交给后端；后端没有做过滤与安全处理，直接把用户输入的字符串，原样拼接到 SQL 查询语句的条件位置，让用户输入直接变成查询条件的一部分。

4.为什么会产生 SQL 注入漏洞

后端将用户可控的输入直接当作 SQL 代码的一部分拼接执行，没有区分「固定查询代码」和「用户输入数据」。攻击者可以通过输入单引号、逻辑命令、注释符等特殊字符，篡改原本的查询逻辑，随意修改、绕过查询条件，因此产生了 SQL 注入攻击的可能。

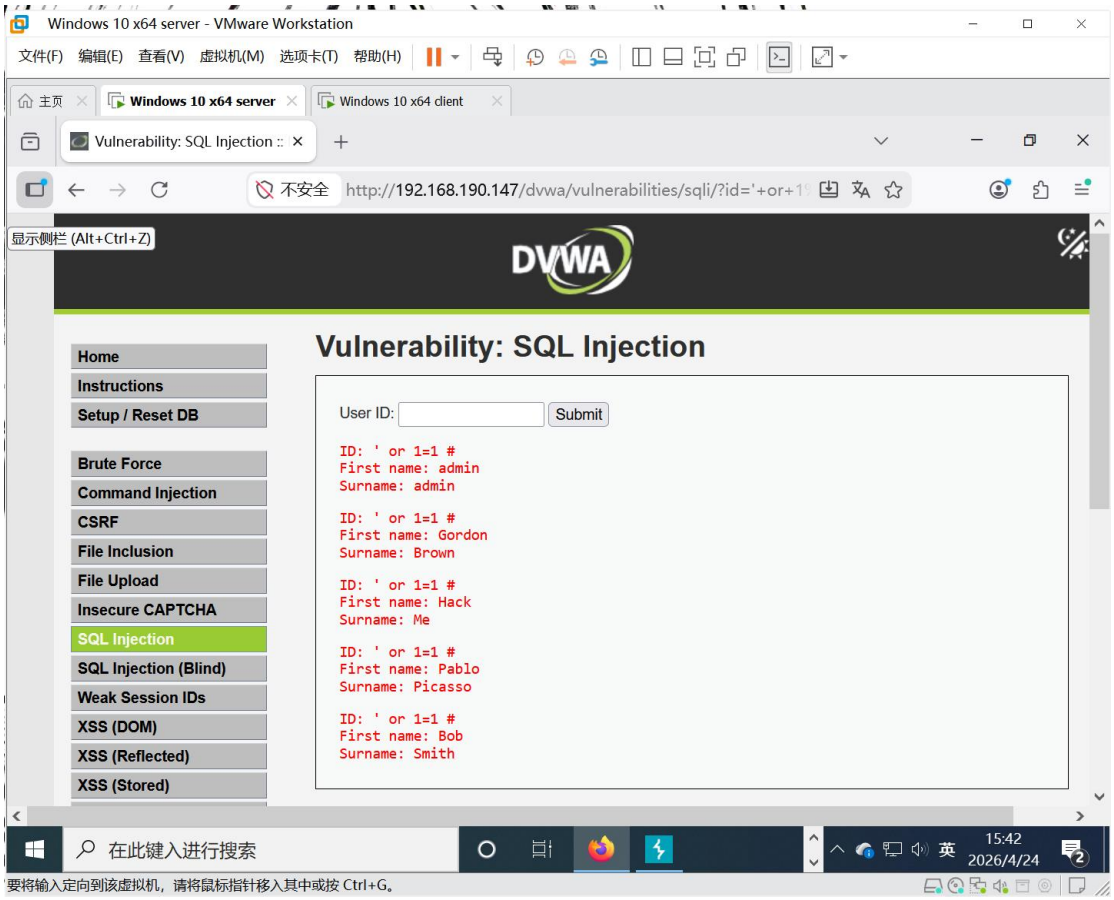
任务 2：DVWA SQL Injection Low 现象记录

正常输入



第一张截图（ID=1）：验证了页面的正常业务逻辑，支撑 “正常查询只会返回单条结果” 的判断。

异常输入



第二张截图（' or 1=1 #）：验证了注入导致的异常结果，支撑 “SQL 语句被篡改，查询结构被污染” 的判断。

1. 实验页面 / 实验目标

实验环境：DVWA 漏洞测试平台，安全等级设置为 Low

实验页面：Vulnerability: SQL Injection 用户 ID 查询页面

实验目标：对比「正常 ID 输入」与「恶意注入输入」的页面返回差异，验证 SQL 注入漏洞的存在

2.普通输入与异常输入的对比

输入类型	输入内容	提交方式
普通输入	1（纯数字 ID）	直接在 User ID 框提交
异常输入	' or 1=1 #（注入载荷）	直接在 User ID 框提交

3.返回结果的差异

（1）普通输入（ID=1）：页面仅返回单条数据，显示 ID 为 1 的用户信息：

ID: 1

First name: admin

Surname: admin

结果完全符合「按 ID 精准查询」的业务逻辑。

（2）异常输入（Payload=' or 1=1 #）：页面不再限定单一 ID，直接返回了数据库 users 表中的所有用户数据，包括 admin、Gordon、Hack、Pablo、Bob 等多条无关用户信息。

4. 为什么这种差异不像“正常查询变化”

正常查询的逻辑是「输入一个 ID，返回一条对应数据」，输入内容改变只会切换单条结果，不会出现批量全表数据。而注入输入后，页面直接突破了 ID 限制，一次性爆出了数据库里的所有用户信息，完全违背了页面原本的精准查询逻辑，是后端未过滤输入、SQL 语句被篡改导致的异常行为，因此不属于正常的查询变化。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

用户在页面的输入框里输入内容（比如 1 或者' or 1=1 #），这些内容会被直接提交给后端程序。后端没有做任何安全处理，直接把用户输入拼接进 SQL 查询语句里，让原本只是“数据”的输入内容，变成了 SQL 代码的一部分，接着数据库对拼接后的语句进行解析、执行，最后把结果返回页面显示。

完整链条：用户输入 → 后端直接拼接进 SQL 语句 → 数据库解析语句 → 执行查询 → 页面回显数据或行为差异。

2. 为什么根因是“结构被污染”而不是“字符本身危险”

单引号、or 这些字符本身并不可怕，它们只是普通符号。真正危险的是：用户输入直接进入了 SQL 查询结构，让数据变成了代码，破坏了原本固定的查询结构，导致数据库把用户输入当成指令执行。所以根因不是字符危险，而是查询结构被用户输入污染、篡改。

3. Low 与 Medium 的差异在哪里

（1）输入方式与前端防护差异

Low 级页面是自由输入框，可以直接敲入带单引号的 payload，比如 ' or 1=1 #，前端没做任何限制，想输什么就输什么。

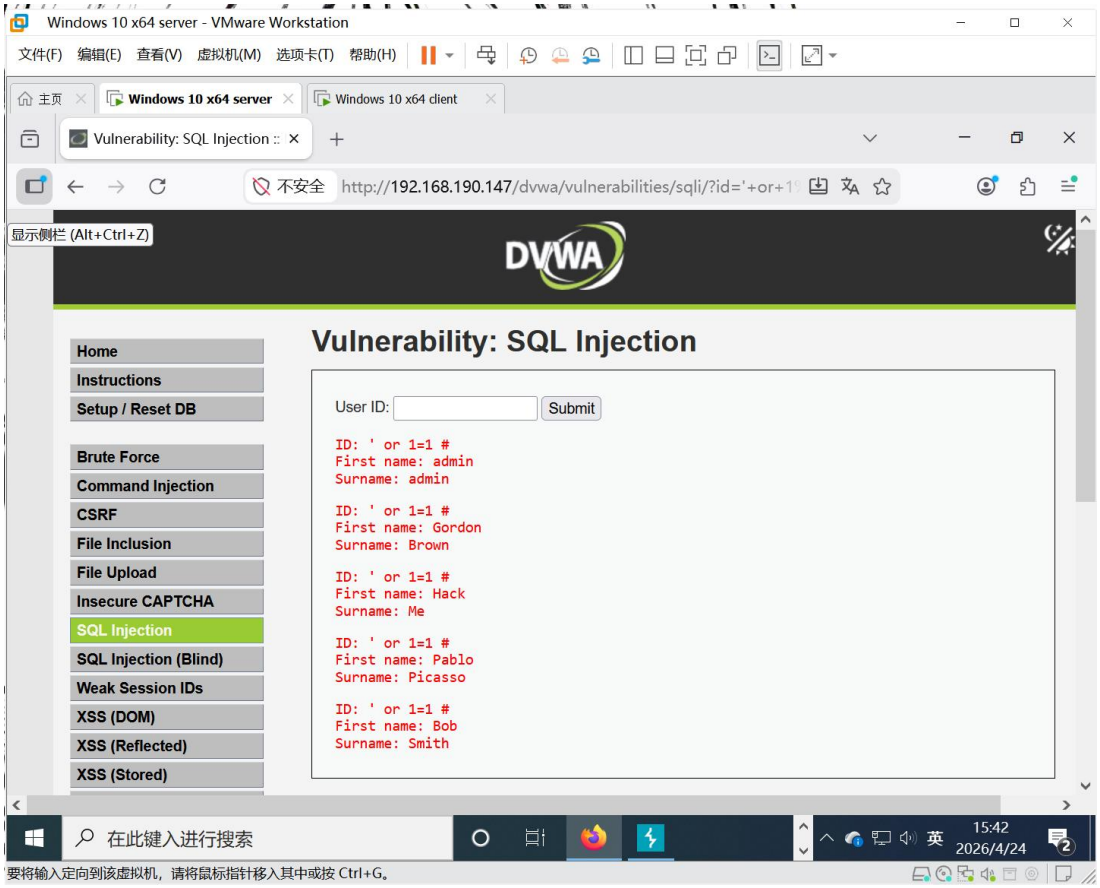
Medium 级就不一样了，页面直接变成了下拉选择框，只能选预设的数字 ID，没法直接输入特殊字符，前端先给输入加了一层限制。

（2）后端防护差异 Low 级后端完全没过滤，用户输入直接就拼进 SQL 语句里了，单引号能直接闭合原来的查询条件，注入起来特别顺。Medium 级后端对单引号这些特殊字符做了转义处理，比如直接输入 ' or 1=1 #，会被转成 \' or 1=1 #，没法再用单引号闭合语句，常规的字符串型注入直接就失效了。

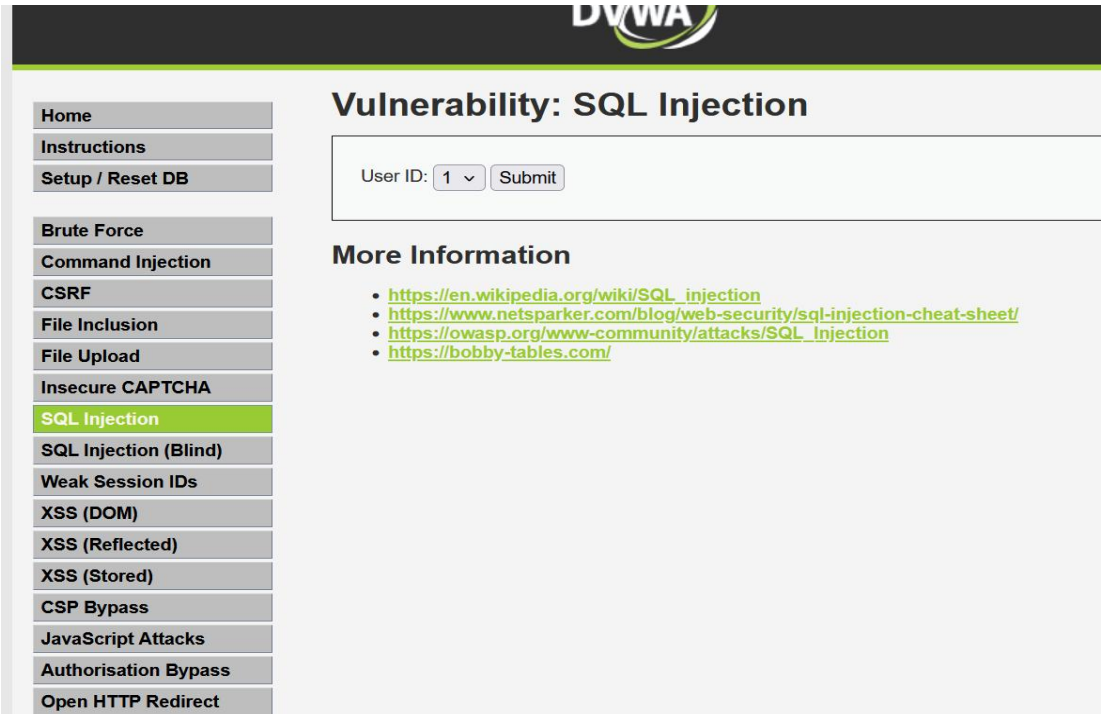
（3）注入利用方式差异 Low 级很简单，直接在输入框里写 payload 就行，如第一张图里的 ' or 1=1 #，提交后页面直接返回了所有用户数据。Medium 级就得多走一步，得用 Burp Suite 抓包改包，绕过前端下拉框的限制，用不带单引号的数字型 payload，比如 1 OR 1=1，提交后才能爆出所有数据，如图三。

（4）核心差异总结简单说，Low 级就是完全不设防的字符串型注入，直接输入 payload 就能篡改查询结构；Medium 级虽然加了前端限制和后端转义，但没防住数字型输入，只能通过抓包改包、用数字型 payload 绕过，注入的门槛明显更高了。

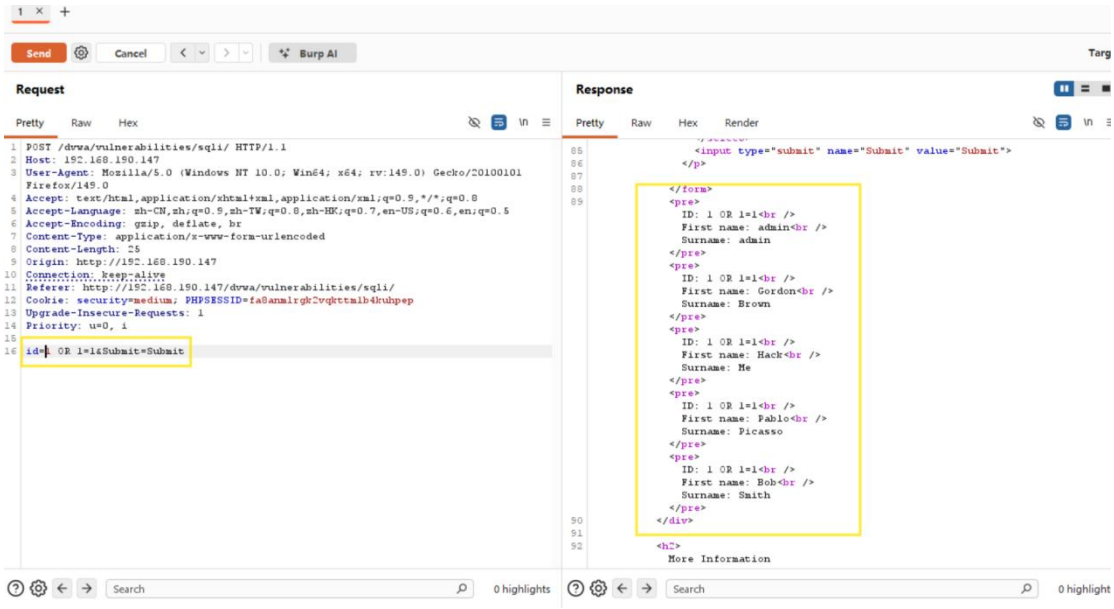
low 图一



Medium 图二



图三 在 bp 里进行注入



4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里

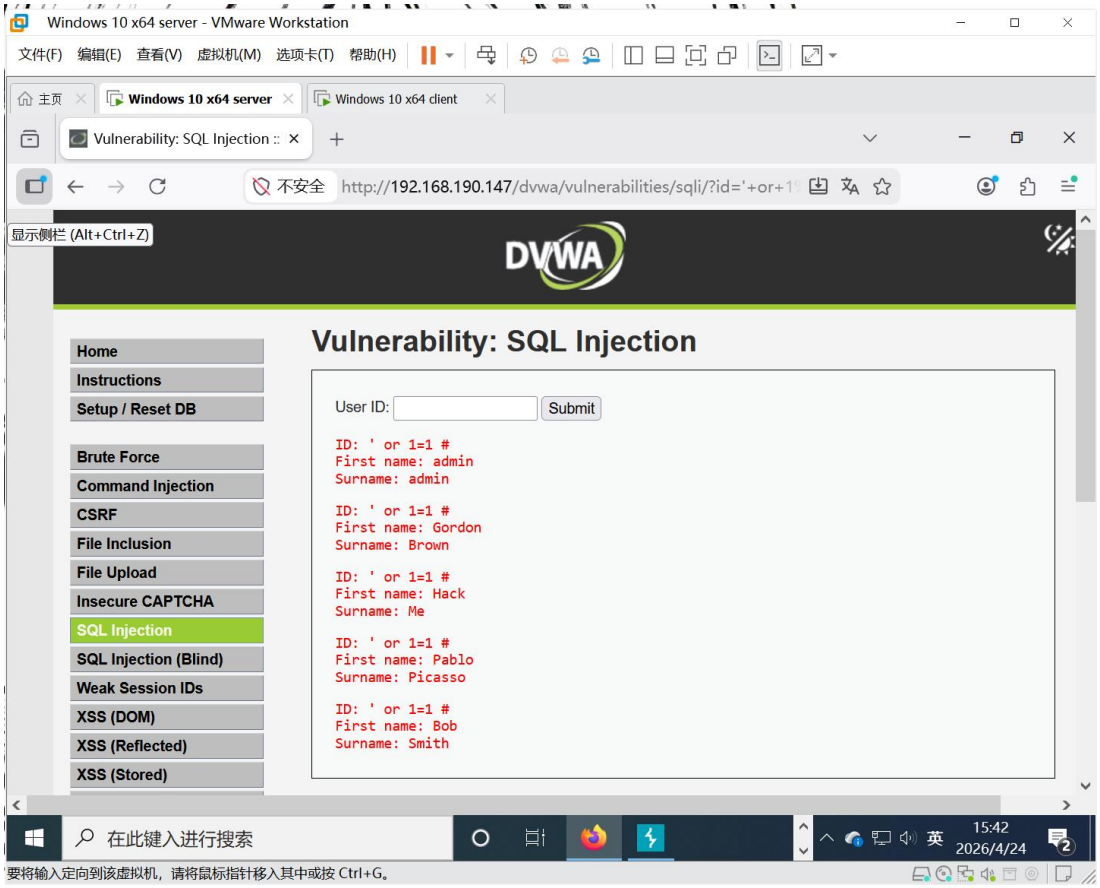
（1）页面回显方式不同普通 SQL 注入会直接把数据库里的查询结果明文显示在页面上。比如我输入' or 1=1 #，页面直接把所有用户的 ID、姓名都打印出来了，能一眼看到完整数据。而盲注不返回任何查询数据，只给我两个状态提示：User ID exists in the database 或者 User ID is MISSING from the database，看不到任何真实的用户信息。

（2）判断注入成功的依据不同普通注入的证据很直接，就是“看到不该出现的全表数据”。我输入恶意语句后，页面直接返回了所有用户信息，说明 SQL 语句被篡改成功了。盲注的证据不是数据本身，而是页面状态的变化。比如输入 1' AND 1=1#时，页面提示“存在”；输入 1' AND 1=2#时，页面提示“不存在”。这种“真假条件对应不同页面反馈”的差异，就是盲注成功的证据。

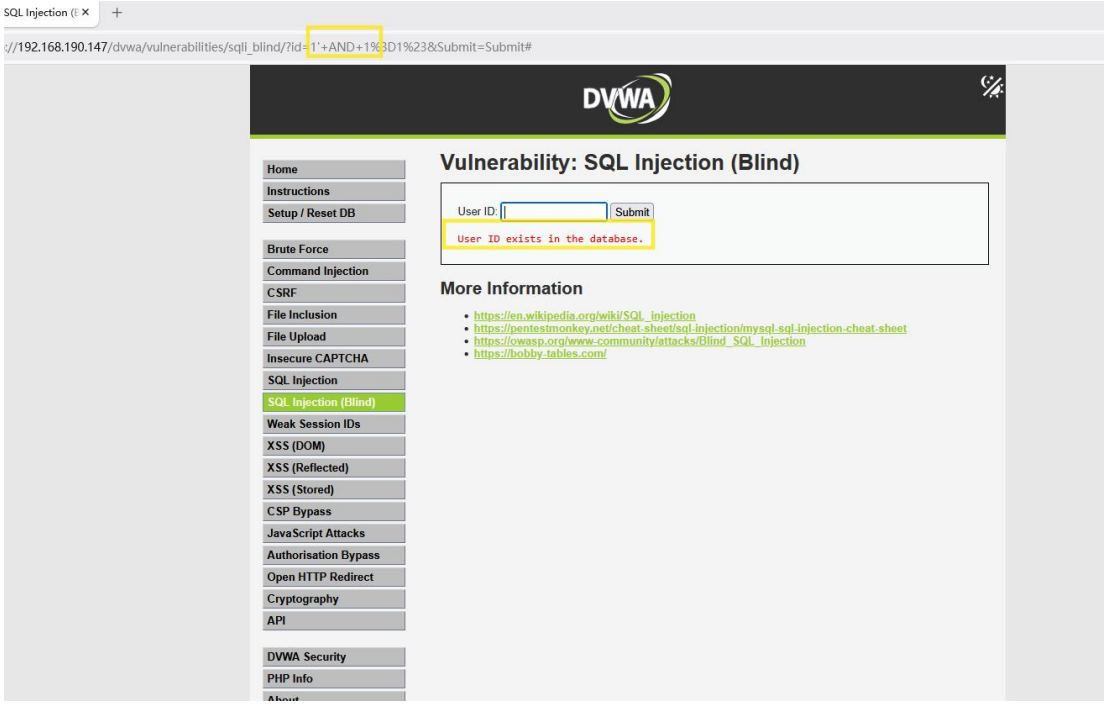
（3）注入利用方式不同普通注入可以直接用' or 1=1 #这类语句，一次性把所有数据都查出来，过程很直接。盲注没法直接看到数据，只能通过构造真假条件，一点点“猜”出数据库的信息。比如通过 AND ASCII(SUBSTRING(version(),1,1))=51#这种方式，逐字符判断版本号，过程要复杂得多。

（4）核心差异总结两者的漏洞根因都是 SQL 语句被用户输入篡改，但表现形式完全不一样：普通注入是“数据明文回显”，盲注是“只有状态反馈，不返回数据”，所以一个靠直接看结果判断，一个靠页面行为差异来推断

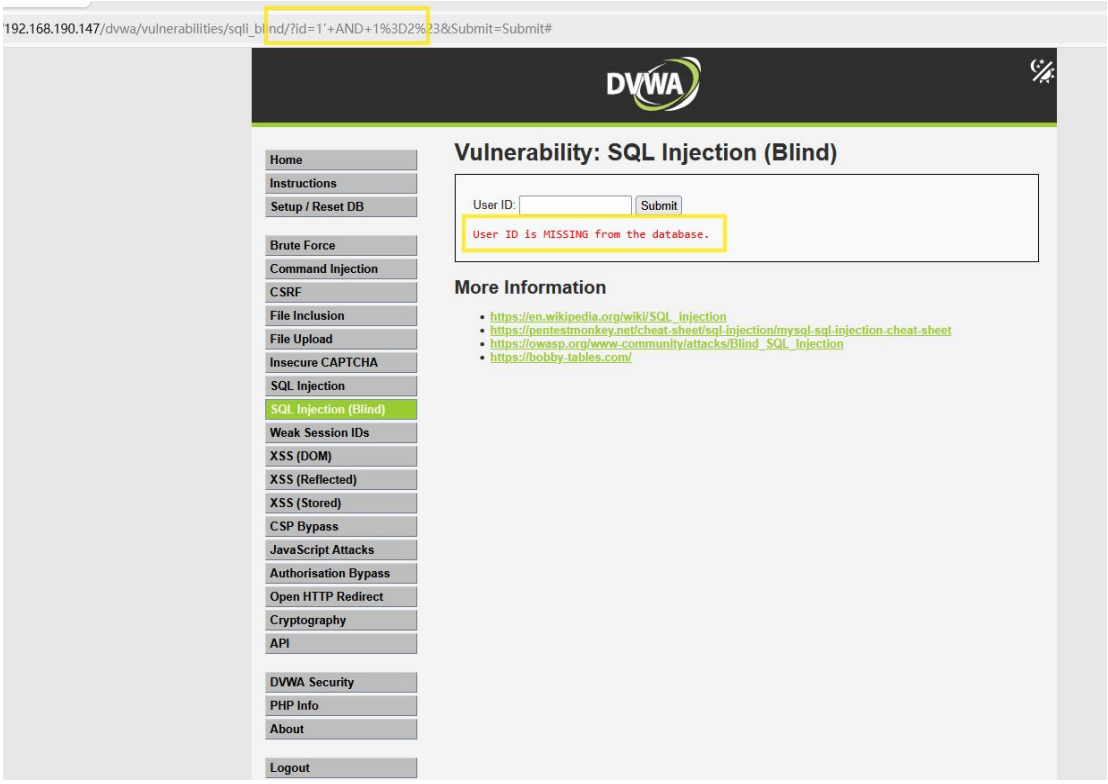
普通注入：输入' or 1=1 #



盲注：1' AND 1=1#



盲注：1' AND 1=2#



5. 为什么盲注和普通注入根因相同，但证据形式不同

它们的根因完全一样：都是查询结构被污染，输入被拼接执行。证据形式不同，是因为页面展示不一样：普通注入页面会回显数据，证据是“看到具体内容”；盲注页面不回显数据，只返回状态，证据是“页面行为的差异”。

强制判断题（必须明确表态）

1.SQL 注入的根因更接近：B

理由：SQL 注入的核心是输入被拼进了不该进入的查询结构，让用户数据变成了 SQL 指令。单引号等字符只是利用工具，不是根因。

2.盲注与普通注入最大差异更接近：B

理由：盲注和普通注入的漏洞原理完全相同，都是查询结构被污染。差异只是页面回显方式不同，导致攻击的证据形式不一样，根因并没有区别。

任务 4：SQL 注入防护清单与解释

1. 参数化查询（预编译语句）

解决的问题：从根本上杜绝用户输入篡改 SQL 语句结构。它会把「SQL 指令」和「用户输入数据」完全分开处理，用户输入只会被当作纯数据，永远不会被解析成 SQL 代码。

控制层级：根因控制层（直接解决“输入污染查询结构”的核心问题）。

局限性：无法替代输入校验和权限控制，也不能防御业务逻辑漏洞；如果实现不当（如动态拼接表名、排序字段），依然可能存在注入风险。

2. 输入校验（白名单校验 / 类型强校验）

解决的问题：限制用户输入的内容格式，提前过滤掉不符合业务预期的恶意输入。比如 ID 查询场景，只允许输入数字；用户名查询场景，只允许字母、数字和有限特殊字符。

控制层级：暴露面控制层（缩小可被利用的输入入口范围）。

局限性：无法作为唯一防护手段，容易被绕过（如绕过黑名单过滤），也无法覆盖所有业务

场景的输入，需要配合参数化查询使用。
3. 错误处理
解决的问题：防止数据库错误信息泄露给攻击者。例如关闭 SQL syntax error 这类详细报错，避免攻击者通过报错信息推断数据库结构、字段名等关键信息。
控制层级：暴露面控制层（减少漏洞被发现和利用的线索）。
局限性：不能阻止注入本身，只是隐藏了漏洞利用的证据；攻击者依然可以通过盲注等方式继续利用漏洞。
4. 最小权限原则（数据库账号权限控制）
解决的问题：限制被注入后的攻击影响范围。例如给 Web 应用使用的数据库账号，只授予查询、修改必要表的权限，禁止 DROP TABLE、DELETE 等高风险权限，也禁止访问系统表。
控制层级：后果范围控制层（降低漏洞被利用后的破坏程度）。
局限性：无法阻止注入攻击本身，只能在漏洞被利用后，减少攻击者能造成的损失。
5. 额外补充：Web 应用防火墙（WAF）
解决的问题：在应用入口处拦截包含常见注入特征的请求（如' or 1=1、UNION SELECT 等），作为额外的防护屏障。
控制层级：暴露面控制层（增加攻击成本，拦截已知攻击载荷）。
局限性：存在被绕过的风险（如变形编码、特殊字符绕过），不能替代参数化查询等核心防护措施，仅作为辅助防护手段。
收口项 1：SQL 注入因果链/防护关系图

环节	内容	因果关系	对应防护控制点
输入点	用户在页面输入 ID、用户名等内容	攻击的起点，所有注入的入口	输入校验（限制输入格式）、WAF（拦截恶意请求）
查询构造	后端将用户输入直接拼接到 SQL 语句中	注入发生的关键环节，结构被污染	参数化查询（将数据与指令分离）
数据库解析 / 执行	数据库将被篡改的 SQL 当作指令执行	攻击生效的环节，执行恶意逻辑	最小权限（限制执行权限，降低后果）
结果差异 / 行为证据	页面返回全表数据、报错信息或行为异常	注入成功的直接证据	错误处理（关闭详细报错，隐藏证据）
根因判断	核心原因是「输入污染了 SQL 查询结构」，而非字符本身	决定了防护措施的优先级	参数化查询（直接解决根因）

收口项 2：学生自评表
1.我本章最清楚的一点是什么？我最清楚的一点是：SQL 注入的根本原因是“查询结构被污染”，而不是输入里的特殊字符本身。比如 Low 级里的' or 1=1 #，真正危险的不是引号，而是引号让用户输入直接变成了 SQL 语句的一部分，篡改了查询逻辑。
2.我最容易混淆的一点是什么？我最容易混淆的是：普通 SQL 注入和盲注的差异到底在哪。一开始我以为它们是两种不同的漏洞，后来才明白，它们的根因是一样的，只是页面回显方

式不同：一个直接给数据，一个只给状态反馈，证据形式不一样而已。
3.我本次作业中哪一段最需要教师复核？我觉得“盲注与普通注入的根因相同，但证据形式不同”这一段最需要老师复核。我怕自己理解错了，把“表现形式差异”和“漏洞本质差异”搞混了，想确认一下这个判断是不是准确。
4.如果重做一次，我会优先改哪一部分？如果重做一次，我会优先把 Low 和 Medium 级的对比分析写得更细一点。现在只是写了输入方式和防护差异，下次我想把两个等级的 SQL 语句变化、payload 绕过的具体过程写得更清楚，让老师一眼就能看到我理解了它们的防护逻辑。
收口项 3：AI 输出审核与人工复核记录
1. 本作业哪些内容使用过 AI 辅助
作业的整体结构框架、各题目的核心得分点梳理，以及部分实验现象的书面表达优化，使用了 AI 辅助整理。
部分关键术语（如“查询结构被污染”）的解释和作业规范的写作方式，参考了 AI 的示范表述。
2. 哪些判断是你自己复核后保留的
（1）关于“SQL 注入的根因是查询结构被污染，而非危险字符本身”的判断，是我结合 Low 级直接注入、Medium 级抓包绕过的实验现象，复核后确认保留的核心结论。
（2）“普通 SQL 注入与盲注根因相同、仅证据形式不同”的判断，是我对照两张注入成功 / 失败的页面截图，结合自己的操作过程复核后保留的。
（3）强制判断题的两个答案，都是我根据实验现象和原理理解，自己确认后最终确定的。
3. 哪些内容你发现 AI 输出不准确并进行了修正
最初 AI 给的 Low 与 Medium 差异描述，把 Medium 级防护写成了“过滤所有特殊字符”，但实际我通过 Burp 抓包测试发现，它只是转义了单引号，数字型输入依然可以绕过，所以我修正了这部分描述，补充了“数字型 payload 绕过”的细节。
4. 你最终如何确认结论和证据是一一对应的
每一个结论都对应了我自己的实验截图：比如 Low 级注入结论对应了全表数据泄露的截图，Medium 级绕过结论对应了 Burp 抓包的响应截图，盲注的状态反馈结论对应了“存在 / 不存在”的提示截图。